Runtime Verification of Robotic Aspects of the Mars Rover

Mattea Fenech

Supervisor: Dr. Christian Colombo

May 2023

Submitted in partial fulfilment of the requirements for the degree of B.Sc. Computing Science.

L-Università ta' Malta
Faculty of Information & Communication Technology

L-Università ta' Malta

University of Malta Library – Electronic Thesis & Dissertations (ETD) Repository

The copyright of this thesis/dissertation belongs to the author. The author's rights in respect of this work are as defined by the Copyright Act (Chapter 415) of the Laws of Malta or as modified by any successive legislation.

Users may access this full-text thesis/dissertation and can make use of the information contained in accordance with the Copyright Act provided that the author must be properly acknowledged. Further distribution or reproduction in any format is prohibited without the prior permission of the copyright holder.

Abstract

The Curiosity Mars rover is one of the most sophisticated successfully deployed systems in the history of planetary exploration. With limited autonomy on board, the ground control team makes most of its decisions. Communication from Earth to Mars may cause messages to take between 4 to 24 minutes to transmit according to the orbital position of the planets at the time of transmission.

Could the rover move autonomously, it would be able to explore the terrain and collect data significantly quicker. However, one major challenge to the increased use of autonomous exploration in such scenarios is the lack of assurances that its behavior will work as expected.

In this dissertation, runtime verification was used to ensure the case study, a simulated Curiosity Mars rover, would not execute any commands which would damage the rover. A separate control system was used to decide the next movements it should take and then be monitored by a built-in monitoring system. The 3 systems communicated between themselves ensuring the next command executed would not damage the rover hence removing the need for ground control to safeguard the rover.

The experimental setup confirmed the effectiveness of the proposed RV system. It demonstrated that the RV system outperformed the traditional method of manual verification by a significant margin, enabling faster and more efficient command verification without relying on a ground control team for each command. This leaves lots of room for implementing a command system for full autonomy to save in the time delay created by having to communicate between Earth and Mars

Acknowledgements

I would like to thank my supervisor Dr. Christian Colombo for his guidance and patience throughout this project. I would also like to thank Mr. Mark Gatland for being of assistance whenever I had a query about his Rover Simulation used in this project. Lastly, I would also like to express my gratitude to my family and friends for their unwavering support throughout the journey of writing my thesis.

Contents

Abstract **i**

Acknowledgements **ii**

Contents **iv**

List of Figures **v**

List of Tables **vi**

List of Abbreviations **vii**

Glossary of Symbols **1**

1 Introduction **1**
1.1 Motivation 1
1.2 Aims and Objectives 2
1.3 Our Approach 3
1.4 Document Structure 3

2 Background **4**
2.1 Runtime Verification 4
2.1.1 Monitors 4
2.1.2 Reactions 5
2.2 Larva 5
2.3 Robotic Operating System 6
2.3.1 RosBridge 7
2.3.2 ROS Monitoring 7

3 Design & Implementation **9**
3.1 Chosen Solution 9
3.1.1 General Approach 9
3.1.2 Rover Control System 11
3.1.3 The Rover System 11

3.1.4 The Verification System 12
3.2 Environment 19

4 Test & Evaluation **20**
4.1 Evaluation Strategy 20
4.2 Results obtained 23
4.2.1 CPU usage per test case 23
4.2.2 Time taken to send and validate commands per test case 23
4.3 Discussion 24
4.3.1 The Verification System Efficiency 24
4.3.2 Running the implementation on the Curiosity Rover 26

5 Conclusions & Future Work **28**
5.1 Revisiting the Aims & Objectives 28
5.2 Critique & Limitations 28
5.3 Future Work 29
5.4 Related Work 30
5.5 Final Remarks 31

A Test Results **33**

B Github Links **34**

List of Figures

Figure 2.1 Example of Larva Property 6
Figure 2.2 Publishing ROS message 7

Figure 3.1 The general architecture of the project 10
Figure 3.2 Block locking implementation 14
Figure 3.3 Obstacle detection monitor model 16
Figure 3.4 Rover velocity monitor model 17
Figure 3.5 Vertical monitor model 18

Figure 4.1 Realistic martian terrain 22

List of Tables

Table 4.1 Results of CPU Usage in Verification System. 23
Table 4.2 Results of Latency Measures. 24

Table A.1 Results of all tests run to collect CPU Usage in Verification System. . . 33
Table A.2 Results of all tests run to collect the Command Validation Latency in Verification System. 33

List of Abbreviations

JVM Java Virtual Machine.

ROS Robot Operating System.

RQT ROS Quality Tools.

RV Runtime Verification.

TCP Transmission control protocol.

WSL Windows Subsystem for Linux.

1 Introduction

Exploration of Mars has been going on since the late 20th century via remotely controlled spacecraft. The data collected from these devices has significantly contributed to the amount of information we know about Mars. This data collection has namely been provided by the Mars rovers, of which all have been controlled remotely from Earth. Therefore, any instructions such as moving the rover or taking photographs had to be given by a ground control team. This has proven to be arduous as sending and receiving messages from Earth to the rover is very time-consuming due to the distance.

For these reasons, if a rover could have built-in software that allowed it to control its own navigation, data could be collected passively. Considering the high costs associated with Mars Rover missions, any failure in such a system could result in the loss of millions of dollars invested in a mission that cannot currently be recovered or repaired. Thus, some form of verification must be carried out to ensure the rover will not damage itself while traversing the Martian terrain.

Testing may be used on the system before deployment, nevertheless, exhaustive testing of the system would be easier said than done as typical testing can never exhaustively confirm every aspect of a system as being safe. This is where runtime verification is introduced as specific elements of the rover system could be monitored in real-time as the rover collects data and prevents any execution from possibly destructive commands.

1.1 Motivation

Research into automating the navigation and data collection of the Mars rovers through the use of RV is not a new concept [1]. Despite this by using ROSmonitoring in conjunction with Larva, the verification process is strengthened, introducing a novel implementation of RV to ensure the rover's safety, thereby mitigating potential risks.

Apart from having to create the automaton for each property under surveillance, the true main challenges that will be faced in this project are primarily related to synchronizing the timing between the different elements in each system used so that execution proceeds in the correct order. Any errors in synchronization could result in commands being accidentally corrupted or skipped, not being monitored by the correct properties or halting the execution altogether.

Further, the computation time of the monitors needed to be kept as minimal as possible as this would slow down the data verification being performed by the rover since decreasing data collection time is one of the main reasons why runtime verifica-

tion is proposed. What's more, is the delay in communication between the systems themselves would cause further overhead and so also needed to be optimized to reduce this as much as possible.

This project, therefore, looks into the viability of this implementation and how efficient and effective it is in comparison to other current implementations of RV being used to control the curiosity mars rover.

1.2 Aims and Objectives

Runtime verification is typically used as a software verification method whereby monitors are injected into a system to verify the execution is following a predefined set of specifications. Therefore, this approach is highly suitable for verifying the autonomous command system being used for the Curiosity rover. The aim of this project is to utilize runtime verification to monitor certain elements in the rover system during execution, in order to allow the rover to navigate autonomously without risking damaging itself. In order to do so finite state machines for each property to be monitored were created.

Applying RV to the system poses a considerable challenge in its own right, however, determining if the implementation was a success is something else entirely. Largely due to the overhead created due to the data collection, communication, and verification that needs to be performed before any command can be executed. Nevertheless, this can be determined by evaluating the time it takes for a command and any monitored data to be transmitted and verified by the RV system and comparing this time to the average transmission time for sending and receiving messages between Earth and Mars.

The main research question at hand is: How efficiently can RV be applied to the Curiosity Mars Rover in order to minimize the time delay caused by human interaction? Through this research question, we will be able to better determine the success of implementing RV for autonomous rover navigation. By conducting various tests we will be able to determine how long it would take to execute a set of commands using an RV system as opposed to being transmitted from Earth manually with the transmission delay.

The main objective of this Final Year Project is to showcase how RV can enhance the Rover's data collection while ensuring its safety. Additionally, any resulting overhead is expected to be minimal compared to the time required for communication between Mars and Earth.

1.3 Our Approach

A case study of the Curiosity Mars Rover implemented by Gatland [2] will be used, to run tests to determine the success of the verification system being built. The automaton created for each property will work on an event-based system. For instance, if a speed change command is created by the control system this data is passed into the Larva system and confirmed against the Speed property, ensuring it is abiding by it. If the command fits within the automaton specified it will be passed on to the ROS system to be executed, otherwise, it is simply disregarded.

Each property will have its own finite state automaton to confirm commands against. Any reactions about any erroneous states encountered will be logged and stopped by Larva. Similarly, reactions will also be logged for commands that abide by the automaton.

Simultaneously the rover control system will sequentially send the next command to be executed waiting to receive the go-ahead from the Larva system before sending the next command. At the same time any relevant ROSmonitor data collected will also be sent to the Larva system too, in order to aid in confirming the validity of the command to be executed is staying within the property.

The various systems will communicate with each other using sockets and will acknowledge when a packet has arrived to ensure that the execution proceeds in a correct and orderly manner in all the systems. The execution flow follows a circular pattern: a command will be created in the command control system, then any respective data needed to confirm the validity of the command will be collected using ROSmonitoring. This information will then be sent to the Larva system for verification. If the command is deemed valid, it will be sent to the ROS system to be executed.

1.4 Document Structure

This report is split into five chapters. In this first chapter, an introduction to the report is given. The second chapter is background research into RV and the tools and technologies used to implement the project. The third chapter explains the design and implementation of the project going into detail about how all the systems work together as well as details on each specific system. Chapter four shows the results of the tests run and an evaluation drawn from the results. Finally, the fifth chapter revisited the initial target goals of the project, describes any limitations encountered during the creation of the project, any possible future works that may be carried out, and similar works to that discussed in this report.

2 Background

This chapter aims to aid the reader in order to better understand the rest of the dissertation.

2.1 Runtime Verification

Runtime Verification (RV) [3] is one of several system analysis methods, used to study a system to check execution is proceeding according to a predetermined set of specifications. Specifications for RV are typically expressed using trace predicate formalisms, such as finite state machines, regular expressions, context-free patterns, and linear temporal logic. This allows for a less ad-hoc approach than other forms of testing, such as for instance unit testing. When formal specifications are provided, monitors are then synthesized from them and inserted in the system by means of instrumentation. RV has several applications, such as security or safety policy monitoring, validation and verification, debugging, testing, fault detection and protection, and behavior modification, such as recovery among many others. It provides a lightweight, non-intrusive, and flexible way to detect and diagnose errors and faults in these systems, making them more reliable and secure.

RV may be used to avoid the complexity of traditional formal verification techniques, such as model checking and theorem proving. By analyzing only one or a few execution traces and by working directly with the system. Thus scaling up relatively well and giving more confidence in the results of the analysis as it avoids the tedious and error-prone step of formally modeling the system.

RV can be made an integral part of the target system, monitoring and guiding its execution during deployment. It typically approaches a system as a black box whereby monitors are added to specific parts of the system to ensure the execution of the section being monitored is proceeding as expected.

2.1.1 Monitors

A runtime verification monitor is a software component that is responsible for monitoring the execution of a system and checking if it satisfies a set of specified properties. The monitor is designed to analyze the events produced by the system during runtime and detect and diagnose any violations of the specified properties. The monitors typically operate as independent entities that interact with the system through a set of interfaces or hooks. They observe the system's behavior and evaluate the observations

against their respective specified properties to determine whether the system is behaving correctly.

The properties that a monitor checks can range from simple assertions about the program's behavior to complex specifications of the system's correctness. The monitor can generate alerts or actions in response to the observed violations. Such as logging an error message, triggering a recovery mechanism, or terminating the system.

2.1.2 Reactions

A runtime verification reaction is a response or action taken by a monitoring tool when a violation of a specified property is detected during runtime. By default, all monitors create a trace log of their actions which in it of itself is a reaction. When a monitor detects that the system is not behaving according to the properties specified, it may take various actions, depending namely on the nature of the violation and the system's requirements. For instance, an RV reaction may involve logging an error message, initiating a recovery mechanism, sending an alert, or stopping the system altogether.

2.2 Larva

Larva [4] is a runtime verification platform, a Larva script is passed into the compiler and monitors for the given system are generated. The language was created to be given specific parameters from which it would then generate monitors used to verify the execution of the system. During the execution reactions are created to log all state transitions occurring.

Larva operates by establishing a collection of functions in the Larva script from which it will know to hook onto during execution. These functions act as the transitions between the states. Larva verifies the validity of the execution sequence of these hooked functions by comparing it to the transition list specified in the Larva file.

For instance, in Figure 2.1 we may see an automaton that is being used to assess the type of sentiment the word being monitored has. Once checked the word can be in one of 2 states, "Happy Word" or "Unhappy Word". During the transitions between the "Check Word Sentiment" and "Happy Word" and "Unhappy Word" respectively, Larva will have hooked onto a function such as "checking_words_sentiment(String word)". This function will perform the necessary verification to determine the sentiment of the word, and a Boolean "True" or "False" will be returned based on the decision of the function. Subsequently, the transition will then take place leading to the "Happy Word" state if "True" is returned and the "Unhappy Word" state if "False" is returned.

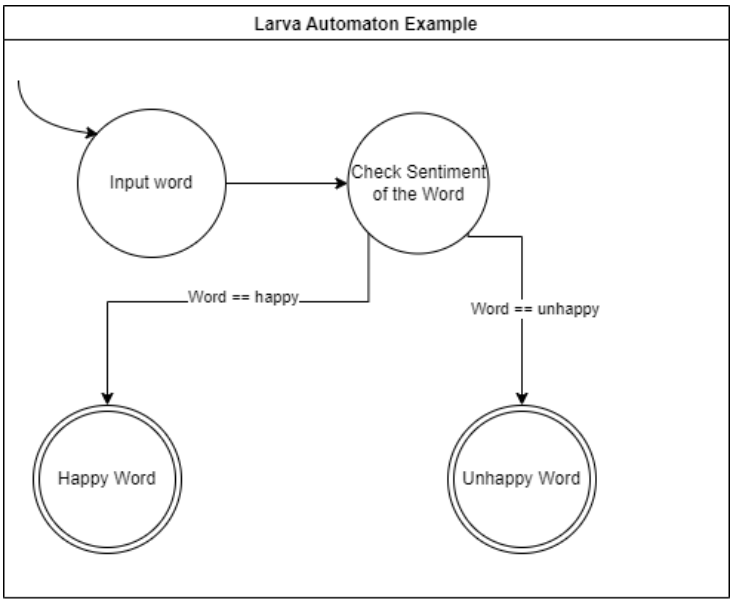

Figure 2.1 Example of Larva Property

2.3 Robotic Operating System

Robot Operating System (ROS) [5] is an open-source control framework used for robotic development allowing for easy development and switching between real and simulated robots. It offers a collection of libraries, tools, and conventions for building complex robot applications such as; hardware driver software, open-source algorithms, debugging and visualization features. It provides a set of software components that can be used to build a wide variety of robot applications, from small mobile robots to large industrial robotic arms. It includes tools for hardware abstraction, communication between different processes, visualization, and debugging.

It was developed to simplify the process of creating and controlling robotic systems by providing a standardized interface and communication protocol between different components of the robot. One of the main advantages of ROS is its modular design, which allows developers to create reusable software components that can be easily integrated into different applications. This makes it easier to develop and test robot applications, and to share code and knowledge within the robotics community.

A ROS system comprises of multiple ROS nodes [6] each node is tasked with performing a certain computation. For instance, controlling the navigation of the robot, computing the data seen by a camera, controlling the movement of a joint, and so on. These nodes are then able to communicate through the use of ROStopics, which are named buses over which nodes are able to send messages. In order for a node to send data to another node it would create a ”publishing” connection allowing it to send the data over. Similarly, a node would be able to collect data from another node by ”sub-

scribing" to it.

2.3.1 RosBridge

The RosBridge is an integrated websocket server built into the ROS framework. Its purpose is to facilitate communication between non-ROS systems and a ROS system. It enables client connections that send a JSON message granting them the ability to subscribe to or publish data to a node.

Figure 2.2 illustrates a typical JSON message used to send data to a node in the ROS system. It exemplifies a scenario where a message is being sent from outside of a ROS system, connecting to a ROStopic that controls the joint holding the mast camera on the Curiosity rover and telling it to move in an anticlockwise direction by 57.3 degrees.

```
{
        "op" : "publish",
        "topic" : "curiosity_mars_rover/mast_cameras_joint_position_controller/command",
        "msg" :
                {
                "data" : -1.0
                }
}
```

Figure 2.2 Publishing ROS message

2.3.2 ROS Monitoring

ROSmonitoring [7] is a framework developed to verify the message exchange in a ROS system at runtime. It is the process of collecting, analyzing, and visualizing data from different components of a robotic system that is running on ROS. Monitoring helps to ensure that the robot is performing as expected and provides insights into the performance of the system. It may be used to collect data from various sources, such as sensors, actuators, controllers, and software modules, and present it in a useful and understandable format.

This data can include information on the robot's status, such as its position, velocity, and acceleration, as well as data on the health of the system, such as CPU usage, memory usage, and network traffic. It offers a variety of tools and techniques, such as command-line tools, GUI-based tools, and web-based dashboards. Some popular ROS-monitoring tools include ROS Quality Tools (RQT) [8] and RViz [9]. Monitoring is an essential part of the ROS development process, as it helps developers to identify problems and optimize the performance of the system. It can also help to detect errors and

faults in the system before they become critical and enable proactive maintenance and troubleshooting.

In the Listing 2.1 a code snippet is provided of what a ROSmonitor would typically look like. A node is being created within the ROS system called "ranges_monitor" as can be seen in line 3 and will be used to subscribe to the ROStopic "/robot/camera/scan". It then periodically checks what data is being generated by this topic specifically, the ranges attribute of this topic. This attribute is an array of all the distances to any obstacles detected around the robot. The "LaserScan" shown in line 11 of the code is a default ROS type which indicates that the data being monitored is a "sensors" message meaning the data being collected is coming from a sensor camera. The "spin" function seen in line 14 is what is making the function periodically check the values of the ranges attribute. In the case of this monitor its use is to print the number of ranges available as well as their actual value.

```
1  def listener():
2      #creating node to monitor ranges
3      rospy.init_node('ranges_monitor', anonymous=True)
4
5      def callback(data):
6          ranges = data.ranges
7          print("Number of ranges: ", len(ranges))
8          print("Ranges: ", ranges)
9
10     # Subscribe to the topic
11     rospy.Subscriber('/robot/camera/scan', LaserScan, callback)
12
13     # Spin until shutdown
14     rospy.spin()
```

Listing 2.1 Code Snippet example of a ROSmonitor

3 Design & Implementation

This chapter will delve into the chosen design approach for implementing the system, providing an explanation for the rationale behind its selection.

3.1 Chosen Solution

The subsequent section will explore the selected solution and provide an explanation for the reasoning behind its selection.

3.1.1 General Approach

The project structure will comprise of 3 primary components, each serving a distinct purpose. The first component is the rover control system, this will assume the responsibility of generating the commands to be executed by the rover. For the purpose of this project, the commands created will consist of moving the mast, changing the rover's speed and creating navigation goals for the rover. These commands will be sent to the verification system via a client connection to the Verification systems Transmission Control Protocol (TCP) server.

The next component encompasses the ROS system, this will be in charge of simulating the environment and rover, instructing the different components on the rover on how to move. Moreover, this system will also be in charge of collecting any data required for validating the commands. The collected data will also be sent to the verification system via a client connection to the same TCP server mentioned above.

Finally, the project will make use of a verification system that will serve to validate all commands sent to it. It will confirm the command's adherence to a set of predefined properties using the data collected from the ROS system in certain cases to verify the commands. Any validated commands will then be relayed to the ROS system for execution via a client connection to the built-in websocket server in the ROS system.

Figure 3.1 shows an overview of the architecture explained above.

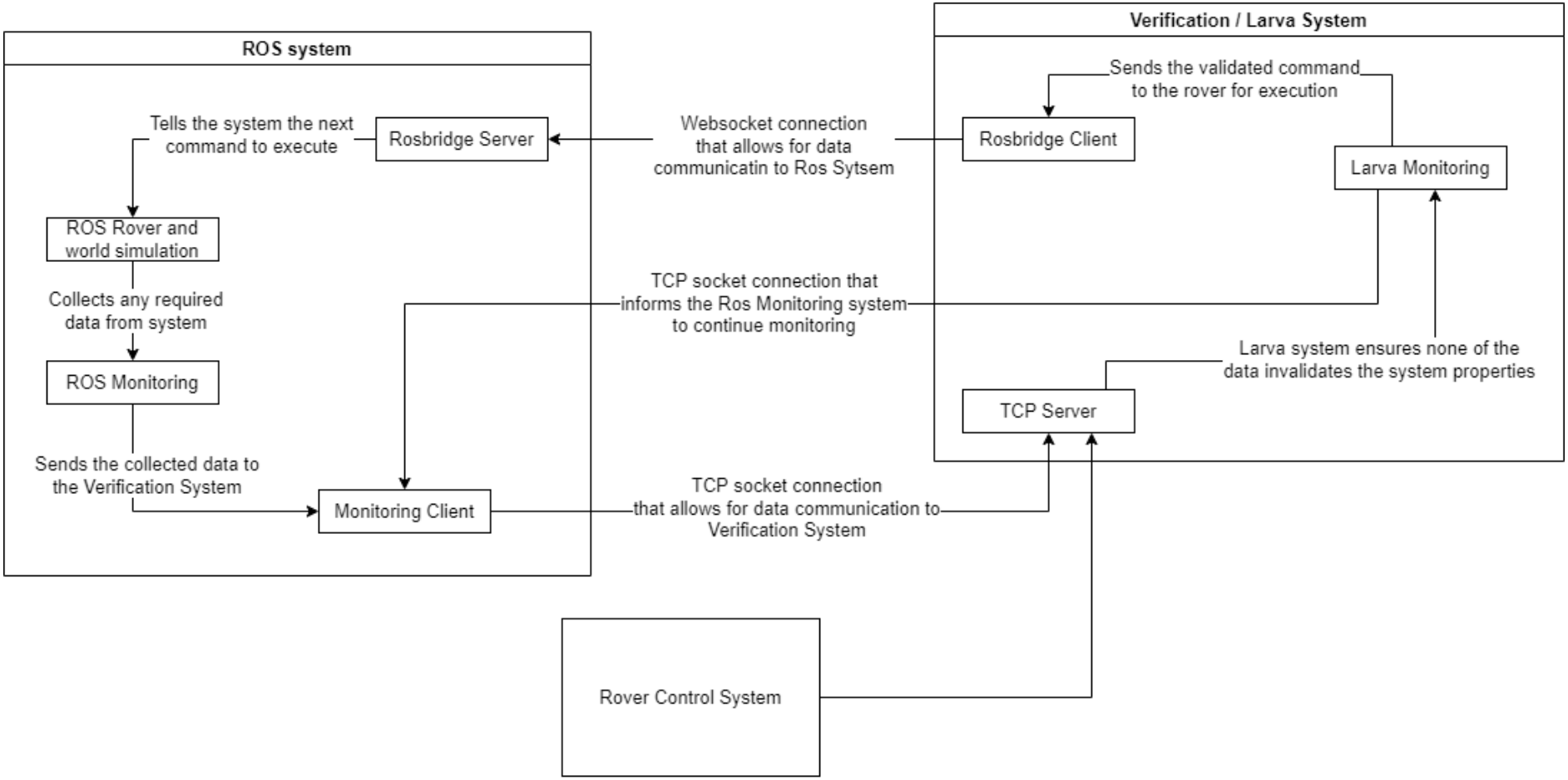

Figure 3.1 The general architecture of the project

In the following section each of the 3 systems listed above will be delved into in further detail.

3.1.2 Rover Control System

This system will be in charge of creating the commands to be executed by the Rover. These commands will subsequently be transmitted to the verification system via a TCP server established on the verification end.

In the case of the project, several hard-coded commands will be created and sent synchronously to the verification system imitating a full-fledged system that would be able to determine the next best command to be executed, such as the next area to navigate to. The implementation of such a system is out of the scope of the project and so a system imitating this functionality will suffice for the testing purposes. The next command will only be sent once the control system receives an acknowledgment from the verification system stating the last command has been verified or rejected.

3.1.3 The Rover System

This will consist of the Rover Simulation created by Gatland [2] as well as the ROSmonitoring system that will be implemented to collect data from the rover and its surroundings. The distinct components that will be involved in implementing this system will now be discussed in further detail.

ROS Simulation

The simulation aspect will consist of several world simulations as well as housing all the code required for simulating the Curiosity rover and allowing the rover to move its parts as well as collect data from the simulation itself.

This specific case study was chosen as the Rover simulation is based on information NASA made publicly available about the Curiosity Mars rover. Moreover, Gazebo is able to compute the physics of the specified world and apply any of the resulting effects on the rover. By utilizing a realistic simulation of the environment and its physical properties, along with the realistic simulated rover, it will be made possible to closely replicate the conditions that the real Curiosity rover likely experiences.

ROS Monitoring

In order to collect data about the properties being monitored several subscribing nodes will be set up to collect data about the angle of the mast, the speed the rover is moving

at and information about the rover's surroundings. This data will then be sent to the Larva system as will be described below.

RosMonitoring will be used as it is a tool built into the ROS framework for monitoring the system, therefore creating the least amount of overhead in data collection. A node can be created and incorporated into the ROS system and made to subscribe to other nodes and collect all the required data.

RosBridge

It will be used to allow the Verification system to connect to the ROS system in order to send any valid commands to be executed by the Rover.

Client connection to the RV System

In order to send the data collected from ROSmonitoring to the RV system a TCP connection will be created. At the ROS system end a client will be established and the data collected will then be transmitted in the form of JSON messages to the server created in the RV system via this connection. A TCP connection will be utilized as the communication between the two systems is relatively elementary and a reliable connection is required to ensure the data arrives safely, both of which are characteristics of a TCP connection.

3.1.4 The Verification System

Within the verification system there will be 3 main threads running:

- **The ROS client thread:** This will be in charge of connecting to the ROSbridge and sending any command the verification system deemed as valid.
- **The Server thread:** This will be in charge of allowing clients to connect from the ROS system to send any monitored data, as well as allow connections from the control system to send any commands that need to be verified before they may be executed on the ROS system. Furthermore. it will parse the received data and store it for the verification thread to then check.
- **The Verification thread:** This will be in charge of taking the data parsed by the server thread and validating it according to the Larva properties, passing it on to the ROS client thread if it is validated. Regardless of if the command was verified or rejected this thread would then relay a message back to the control system saying whether the last command was verified or not and that it may move on to send the next command requiring verification.

Accessing the Shared Resources

The main challenge that will be faced in this project is synchronizing the execution flow of both; all the systems in play as well as all the parts within a given system. Specifically, this issue arises within the Verification system, due to the 3 threads in this system needing to run concurrently. Measures will need to be taken in order to assure no race conditions occur, namely between the server thread which would be writing to the shared variables, and the verification thread which would be reading from the shared variables. If these 2 threads are not synchronized this would likely result in a data race, leading to inconsistent data retrieval by the verification thread. In order to deal with this synchronized blocks will be used to restrict when each thread may access these variables.

Global variables will be abstracted inside of a class and these will store the data collected from the Rosmonitoring data and commands sent to the Verification server. When the verification thread requires read access from these variables to verify the validity of the command to be executed against the automaton, it will attempt to enter into a synchronized lock or wait until it is available. Once available it will enter into a synchronized block section and have sole access to these variables. Once it is in this critical section it may then check the commands against the properties set in the Larva monitoring section. Simultaneously if the server thread were to try to obtain the lock during this time it would be blocked from doing so and would have to wait till the lock is available to then acquire it.

Similarly, when a new message arrives in the server the client handler, will wait till it may acquire the lock. Once the lock has been acquired the thread will then enter its critical section, it will then parse the received JSON message and overwrite the shared variables accordingly.

The use of the synchronized blocks will allow safeguarding against data overwriting, which could cause corrupted commands to be verified and so making the whole verification system redundant. By taking this approach to implementing the multi-threaded system we will have a more granular control of the system since we could better control the order in which the threads acquire access to the variables as well as when. In Figure 3.2 we may see an illustration of how the Verification system will deal with allowing the threads to enter into the blocked section.

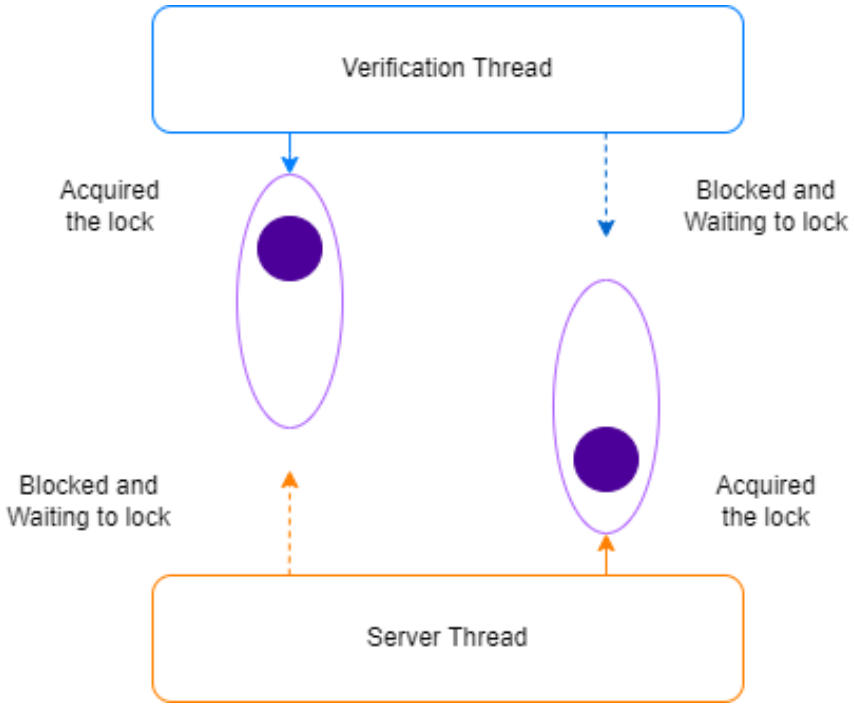

Figure 3.2 Block locking implementation

Alternative Solution for the Shared Resources

Another solution that may have been implemented would have been to create a class of global variables where the class held the master lock to access the shared variables and would decide which thread would get access to the variables next. The class would have several functions that would lock the variables for the thread that is currently accessing them. Subsequently, if the other thread required the variables at the same time it would have to wait till the lock was released and they are available.

Although this implementation would have been simpler to implement as the functionality of choosing which thread would next acquire the variables would be solely decided by the variable's class the implementation that has been decided upon was chosen due to having a more granular control of the lock. Moreover, the approach just explained would likely create more overhead than the chosen approach due to the fact that a single class is in charge of deciding which thread will acquire the lock to the variables next.

Server connection to the ROS System

A TCP server will be created to allow the ROSmonitoring system to establish a connection to send the collected data, along with the Rover control system to connect and send the command to be verified. The data will be sent as a JSON object which is parsed and then verified using the Larva monitoring system as explained below.

The Larva Monitoring System

Another challenge the project presented was implementing the Larva verification properties which will be described in the following section.

The 3 main properties that will be monitored in the Rover system are; its navigation, its speed, and the mast angle. The main focus of this project is to verify the rover does not damage itself during navigation and that it does not collect any redundant data. These specific properties were chosen as monitoring the speed and any obstacles surrounding the rover will help in preventing it from crashing into any obstacles or flipping over. Moreover, the monitoring of the mast joint will help to prevent it from collecting any useless data such as capturing images of parts of itself.

These properties will be described in further detail in the following.

The Obstacle Detection Property

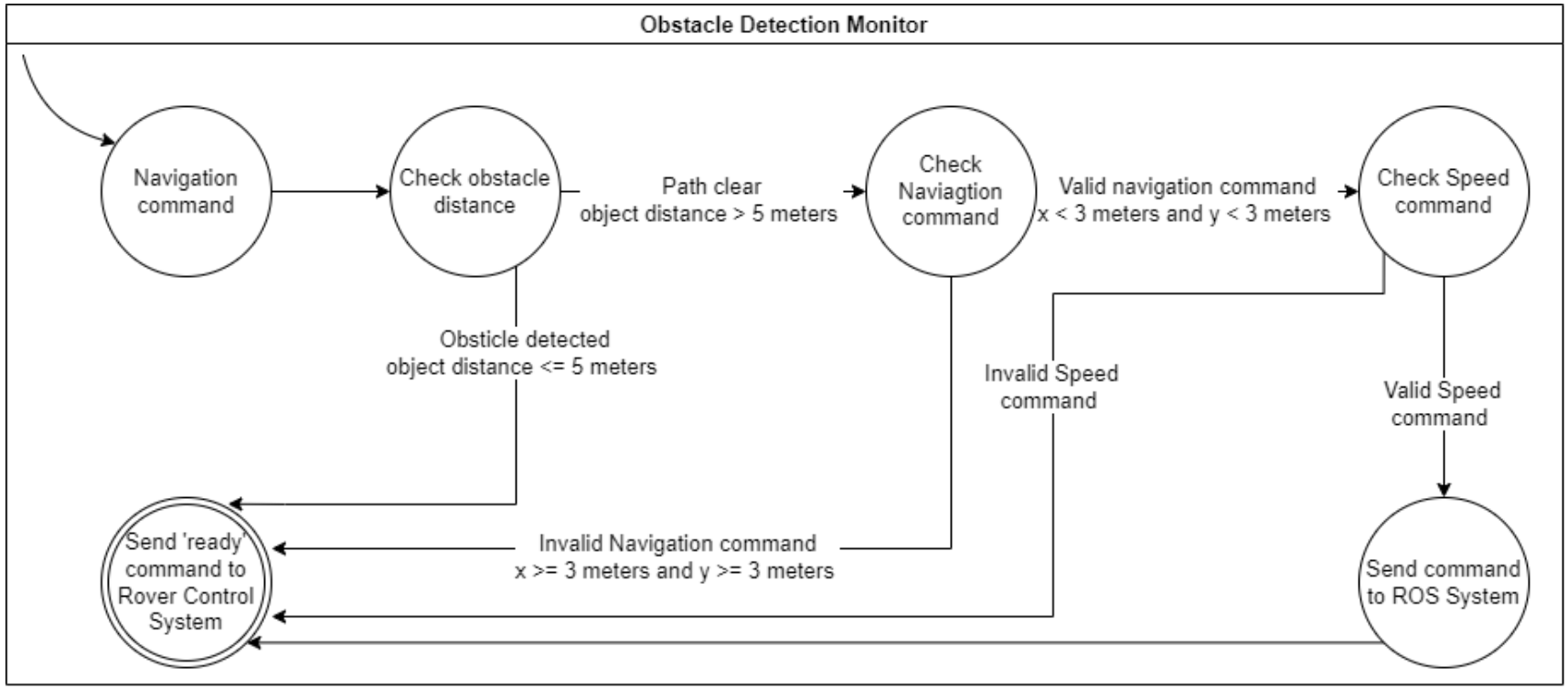

Figure 3.3 Obstacle detection monitor model

As can be seen in Figure 3.3, this property is implemented to protect the rover from colliding with any objects in its surroundings. To realize this property the distance to any objects surrounding the rover is collected using several laser scanners located all around the rover. This data will then be validated in Larva, given no object is closer than the 5-meter safety distance set, the monitor will then move on to validate the navigation command. If no obstacle is detected within a reasonable distance the monitor will then move on to validate the navigation command. Here the monitor will confirm that the ”x” and ”y” navigation goals are smaller than 3 meters, again this will be implemented to restrict the risk of the rover colliding with anything in its surroundings. If this command is verified, the speed command will then be checked for approval against its own automaton. If all of these criteria are not met the command will be discarded as it would not be safe for the rover to move as a collision may occur. Otherwise, the command will be passed onto the ROS system to be executed. Regardless of if the command is validated or not a message will finally be sent to the Command system telling it that it may send the next command for verification.

The Rover Speed Property

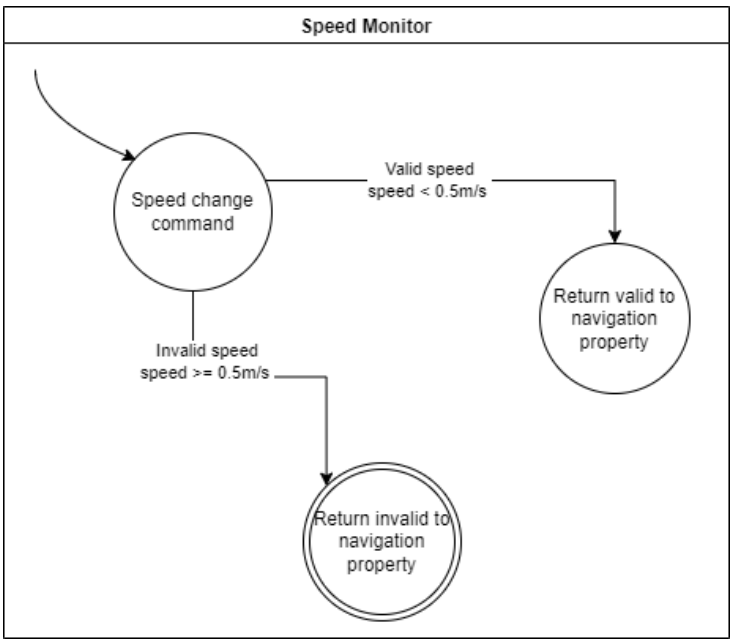

Figure 3.4 Rover velocity monitor model

This property will be implemented to limit the speed the rover moves at as from initial testing it was found that if the rover moved too fast this could result in the rover flipping over due to the momentum exerted by the movement of the rover overcoming the Martian gravity. Figure 3.4 depicts the automaton of this property.

The speed change to be executed is checked in the Larva monitor, if it is smaller than 0.5 meters per second then the speed command will be verified as a valid speed

for the rover to move at. If the speed is invalid then the speed command along with the navigation command will be discarded and the command system will be advised to move on.

The Mast's Movement Property

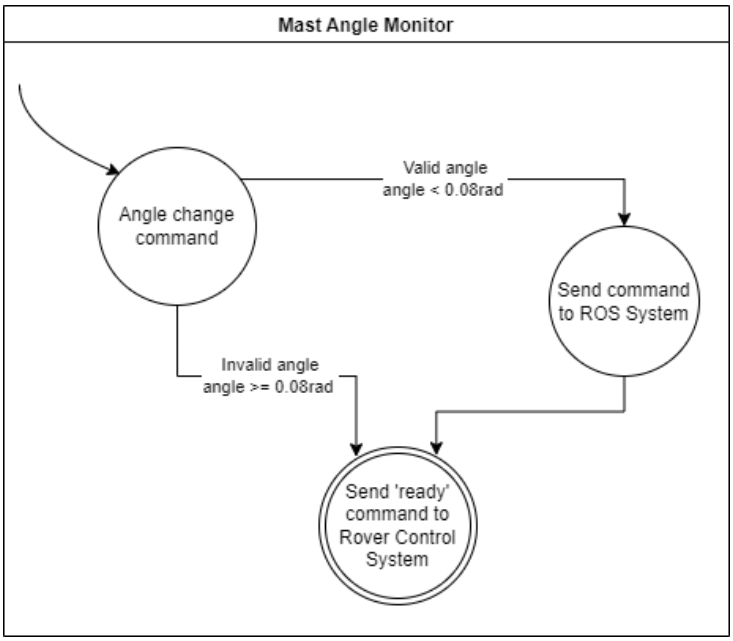

Figure 3.5 Vertical monitor model

This property will be implemented to limit the motion of the mast so that it does not look in a downward direction. Since the Curiosity rover's main objective is data collection viewing its own chase would be a redundant functionality and so this restriction would not have any negative impact on the data collection aspect indeed it would help filter out useless data. In ROS a negative angle would indicate the mast is turning in an anticlockwise direction, while a positive one would mean its turning in a clockwise direction. 0.08 radians is the maximum position the joint could extend to that would not result in the Navcam viewing itself.

The command to be executed is checked in the Larva monitor, if it is smaller than 0.08 radians, then the command may be passed on to the ROS system to be executed. Once the command has been sent to the ROS system another message is relayed to the rover control system to signify that the next command to be executed may be passed to the Larva system. If the command fails to meet the requirements of the model the command is ignored but a message is still sent to the rover control system so that the next command to be checked may be sent. Figure 3.5 illustrates the representation of this property in the form of an automaton.

3.2 Environment

Three main bodies of code will be implemented for the project. The Rover Simulation and ROSmonitoring will be implemented on Windows Subsystem for Linux (WSL) on Ubuntu 20.04 using ROS noetic and Python, the Larva System will be implemented on Windows 11 using Java, Larva, and AspectJ, and the Rover Control System will be implemented using WSL on Ubuntu 20.04 and Python. A high-level implementation of this may be seen in Figure 3.1

4 Test & Evaluation

Throughout the implementation of this project, a monitoring system was created to improve the efficiency of collecting data in a safe manner without requiring human interference. This chapter seeks to test this with several test runs of the entire system with a number of different command sets for the properties being monitored.

Each command test set had a total of 11 commands, a constant amount of commands was used across all the tests so that the CPU usage of each test could better be compared in the end. 5 total command sets were set up each running repeatedly for 3 executions for a total of 15 tests. This approach was taken for testing to ensure a sufficient range of results. In creating the tests I tried to vary the commands between each test case as much as possible aiming to maximize the variety in the results of the CPU usage and time delay.

Not much information is given on how scientists at NASA plan out the navigation goals of the rover and how to move its parts, other than, they navigate to the next place they believe would be best to collect data from and move the cameras in a way that would aid in doing so. Therefore I tried to mimic this behavior based on past navigation goals I had tested on the rover in the beginning stages of the project.

4.1 Evaluation Strategy

In order to test the success of the RV systems implemented the CPU power required to run the Larva system was recorded and the latency of sending and verifying the commands was noted from the following command sets under scrutiny:

- **Command Set 1:** The command system will attempt to send commands that will; move the mast to 0 radians, move the rover forward by 1 meter at 0.01 meters per second, turn right by 90 degrees, and move by 1 meter at 0.1 meters per second, turn left by 90 degrees and move by 2 meters at 0.15 meters per second, move the mast by 0.2 radians clockwise, move forward by 2 meters at 0.02 meters per second, move another meter forward at 0.21 meters per second, turn right by 90 degrees and move by 1 meter at 0.03 meters per second, move forward by 2 meters at 0.05 meters per second, move the mast to 0.03 radians, and finally move 2 meters forward at 0.02 meters per second.

- **Command Set 2:** The command system will attempt to send commands that will; move the mast to 0.02 radians, move the rover forward by 2 meters at 0.05 meters

per second, move another meter forward at 0.1 meters per second, turn by 180 degrees and move 3 meters at 0.02 meters per second, move the mast to 0.1 radians, readjust the mast to 0.01 radians, move straight by 2 meters at 0.02 meters per second, move the mast again to 0.35 radians, turn right by 90 degrees by 2 meters at 0.2 meters per second and turn right by 90 degrees again by 2 meters at 0.7 meters per second.

- **Command Set 3:** The command system will attempt to send commands that will; move forward by half a meter at 3 meters per second, move the mast to -0.2 radians, readjust the mast to 0 radians, turn right by 90 degrees and move by 1 meter at 0.01 meters per second, move forward by 3 meters at 0.1 meters per second, turn left by 90 degrees again and move by 2 meters at 0.7 meters a second, move forward by 2 meters turn right by 90 degrees and move another 2 meters at 0.2 meters per second, move the mast to 0.01 radians, move forward by 1.5 meters at 0.5 meters per second and lastly move forward by 7 meters at 0.2 meters per second and finally by 2 meters straight at 0.2 meters per second.

- **Command Set 4:** The command system will attempt to send commands that will; turn the rover right and move by 2 meters at 0.3 meters per second, move the mast to 0.02 radians, move forward by 1 meter at 0.01 meters per second, move anther 2.5 meters forward at 0.1 meters per second, move the mast to 0.5 radians, turn right by 90 degrees and move by 2 meters at 0.07 meters per second, move forward by 2 meters turn right by 90 degrees and move by 2.5 forward at 0.2 meters per second, move the mast to 0 radians, move forward by 1.5 meters at 2 meters per second, move forward by 5.7 meters at 0.2 meters per second, and another 5 meters at 0.02 meters per second.

- **Command Set 5:** The command system will attempt to send commands that will; move the mast to 0.8 radians, turn to the right and move by 2 meters at 0.3 meters per second, move the mast to -0.02 radians, turn right by 90 degrees and move by 1 meter at 0.15 meters per second, turn left by 90 degrees again and move by 2.5 meters at 2 meters per second, move the mast to 0 radians, move forward by 1.5 meters at 0.4 meters per second and move another 7 meters forward by 2 meters per second, move the mast to 0 radians, move straight by 1.5 meters at 0.4 meters per second and finally move forward by another 7 meters at 0.2 meters per second.

The Martian terrain that the tests were run on was a simulated world created using information NASA made publicly available and collected using the Mars rovers. Hence, the simulated terrain closely resembles the condition the Curiosity Rover likely

experiences ensuring a more precise RV system likely to protect the rover from endangering itself. In Figure 4.1 we may see the rover on the terrain.

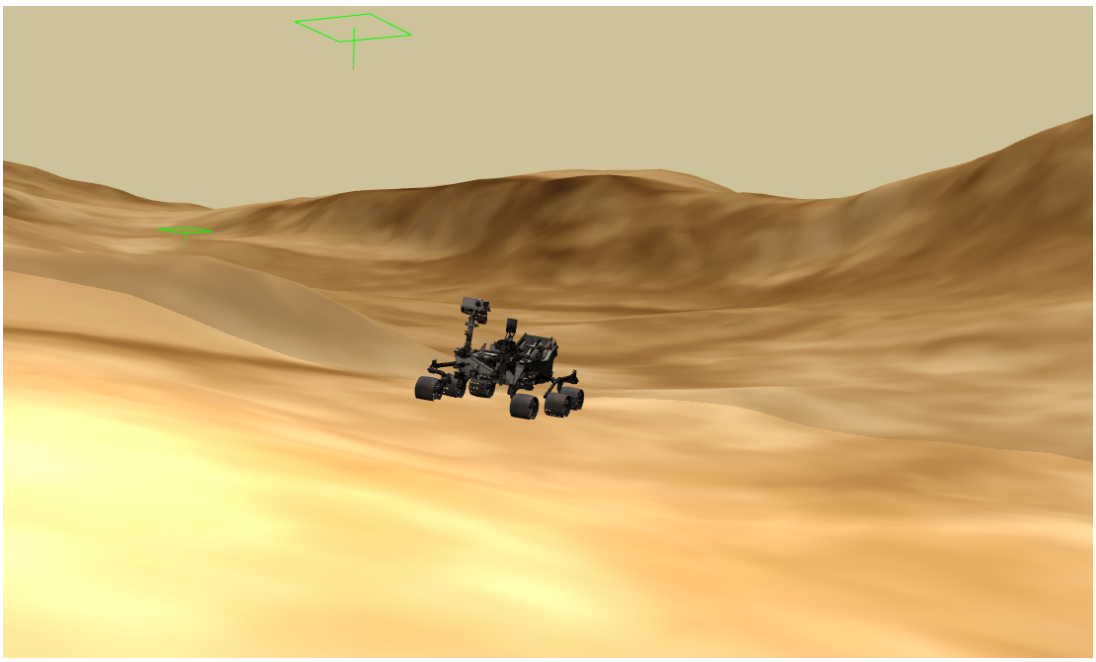

Figure 4.1 Realistic martian terrain

4.2 Results obtained

The evaluation of the system was run on an AMD Ryzen 5 3600 6-Core Processor with a 3.60 GHz clock speed and 16 GB of Ram.

4.2.1 CPU usage per test case

3 tests were run per test case and the average and peak values of the CPU usage were recorded and tabulated. The CPU usage was recorded using the "OperatingSystemMXBean". This is a management interface for the Java Virtual Machine (JVM) that allows certain system properties, such as the amount of CPU power required to run a program to be collected.

The values obtained for the CPU power the Verification system consumed from the tests run may be seen in Table 4.1

Table 4.1 Results of CPU Usage in Verification System.

Test Cases	CPU Usage (%)	
	Peak	Mean
Test 1	3.26	2.45
Test 2	2.61	1.62
Test 3	2.13	1.35
Test 4	4.50	3.90
Test 5	4.47	2.86

4.2.2 Time taken to send and validate commands per test case

3 test were run per test case where the average latency and peak latency was calculated per command sent. The latency recorded was the time it took in seconds to send a command, verify it on the verification system and then send a reply to the command system verifying or rejecting the command.

The results obtained from these tests may be seen in Table 4.2

Finally, we may conclude that the average CPU usage used by the verification system was: 2.44% and the average latency for validating the commands across all the tests was 0.75 seconds.

Table 4.2 Results of Latency Measures.

Test Cases	Command Validation Latency Measures (seconds)											
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11
Test 1	Peak	4.73	0.11	0.19	1.01	0.20	0.99	1.99	0.19	0.18	0.19	0.81
	Mean	3.21	0.09	0.17	0.86	0.16	0.95	1.60	0.11	0.15	0.14	0.69
Test 2	Peak	4.23	0.09	0.19	1.04	0.15	0.08	0.18	0.12	0.15	0.09	0.16
	Mean	2.00	0.08	0.10	0.88	0.12	0.04	0.12	0.07	0.12	0.04	0.15
Test 3	Peak	1.29	0.11	0.18	0.21	5.47	0.17	0.07	0.21	0.09	0.19	0.18
	Mean	0.81	0.08	0.14	0.14	2.34	0.14	0.05	0.15	0.07	0.17	0.11
Test 4	Peak	3.59	0.15	0.18	0.72	0.15	0.13	0.14	0.19	0.13	0.20	0.19
	Mean	1.91	0.11	0.11	0.65	0.09	0.06	0.09	0.13	0.07	0.12	0.13
Test 5	Peak	2.01	0.20	0.17	0.11	5.83	0.13	0.17	0.19	0.17	0.13	0.14
	Mean	1.76	0.14	0.10	0.09	5.71	0.06	0.11	0.18	0.16	0.07	0.06

4.3 Discussion

The two main topics to be discussed in the discussion are how efficient RV is in comparison to human verification and the possibility that the processor on the Curiosity rover would be able to compute the RV implementation created in this project.

4.3.1 The Verification System Efficiency

In order to best compare the verification time using RV versus human verification the time it takes for a message to be transmitted from Earth to Mars will be calculated using proportional relationships.

The average distance between Earth and Mars is 225 million kilometers [10]. At their closest, they are 61 million kilometers apart and a message takes roughly 4 minutes to transmit one way, at their furthest they are 401 million kilometers apart and a message may take 24 minutes to transmit one way.

Let's assume that the time taken to transmit a message is directly proportional to the distance traveled. Although untrue due to the elliptical paths that Earth and Mars are on since only a rough estimation is needed a direct relationship will suffice.

We may then set up a proportion to estimate the time it would take to send a message from Earth to Mars:

$$\frac{401 \text{ million km}}{24 \text{ minutes}} = \frac{225 \text{ million km}}{x \text{ minutes}}$$

To find the value of x, we can cross-multiply and solve for x:

$$(401 \text{ million km}) \cdot (x \text{ minutes}) = (225 \text{ million km}) \cdot (24 \text{ minutes})$$

Dividing both sides of the equation by 401 million km:

$$x \text{ minutes} = \frac{5400 \text{ million km} \cdot \text{minutes}}{401 \text{ million km}}$$

Simplifying the equation:

$$x \text{ minutes} \approx 13.46 \text{ minutes}$$

Therefore, it would take approximately 13.5 minutes to transmit a message one way with a distance of 225 million kilometers between Earth and Mars. Further, it would take roughly 27 minutes for the ground control team to receive data from the Mars rover and send a command they deem fit for execution.

In order to better compare the efficiency of the RV system we will scale the results obtained from the testing section to be more comparable to what the values would realistically look like had they been computed on the Curiosity rover.

The Curiosity rover has a RAD750 Single Board Computer [11] with 200 MHz clock speed and 256 MB of Ram. To determine how many times faster the computer is compared to the rover's processor the ratio of the clock speeds between the 2 computers will be calculated. This value might not be highly accurate due to the 2 computers having significantly different architectures however it will indicate a reasonable representation of the difference in speed between the two computers since the tests could not be run and tested on the actual rover's processor.

To find the ratio between the 2 computers we may set up a ratio between the clock speed of the first computer and that of the second computer:

$$\text{Ratio of Clock Speeds} = \frac{3.6 \text{ GHz}}{200 \text{ MHz}}$$

Simplifying the ratio:

$$\text{Ratio of Clock Speeds} = 18$$

Therefore we are able to conclude the computer the tests were run on was approximately 18 times faster than the Curiosity rover's processor, thus we may calculate the average time it would take to validate an arbitrary command as follows.

$$\text{Average latency to Validate Command on the Curiosity rover} = 18 \times 0.75 \,\text{seconds}$$

$$\text{Average latency to Validate Command on the Curiosity rover} = 13.5 \,\text{seconds}$$

Hence, if the Latency value is adjusted to reflect the actual computation time it would take to run on the rover's processor, we may observe a notable improvement in reducing the time delay through the use of the Verification system.

In order to effectively demonstrate the extent of the improvement brought about through using the RV system compared to the use of command validation by a ground control team, we may calculate the processing ratio that would be needed for an average verification latency to last 27 minutes (1620 seconds).

$$\text{Ratio} = \frac{1620 \,\text{seconds}}{0.75 \,\text{seconds}}$$

Simplifying the ratio we get:

$$\text{Ratio} = 2160$$

Thus, in order for the Verification system implemented to be less efficient than manually validating the commands the Rover processor would have to be at least 2160 times slower than the computer the tests were run on.

4.3.2 Running the implementation on the Curiosity Rover

The setup of the Curiosity rover compared to the system that the testing occurred on is noticeably less computationally capable. That considered on average the CPU usage required for running the test was 2.44%. Thus using the same scaling ratio calculated earlier, on average the CPU usage required to run just the Verification system on the

rover would be as follows:

$$\text{Scaled CPU Usage} = 18 \times 2.44\,\%$$

$$\text{Scaled CPU Usage} = 43.92\,\%$$

Requiring around 44% of the rover's processing power to run the Verification system alone would not be a realistic implementation, due to the processor having other tasks it is responsible for. For example, computing how the different parts of the rover need to move. Due to this, in order for the RV system to be feasibly utilized on the rover, it would require substantial optimization. However such optimization would better be implemented on the actual architecture used by the rover considering the significant difference between the computer the tests were run on and that found in the rover.

Further to this, it is not possible to confirm the feasibility of running the entire system on the rover due to a number of reasons. The command system was not properly implemented, since several hard coded commands were created for the purpose of testing rather than creating a full system capable of generating commands optimised to move to the next best place to collect data. It was not created as it was not a requirement of the project however the lack of implementation of such a system resulted in impeding the possibility of gathering results such as CPU usage which would be required to be able to determine if all the systems implemented in this project could run on the rover. Similarly, the Curiosity rover does not make use of a ROS implementation to control the rover's actions. The inability to calculate the processor consumption of these 2 systems resulting limited our ability to conclude a definitive answer of whether the rover would be capable of running all these systems together.

5 Conclusions & Future Work

Throughout the implementation of this project, 3 systems were implemented to create a Curiosity rover with a command-creating system verified using a verification system using RV. This work was then evaluated to establish a conclusion to the research question stated at the start of the project which will be summarised in this chapter.

5.1 Revisiting the Aims & Objectives

This project presented a new way of how the Curiosity rover can navigate autonomously without endangering itself through the use of RV using ROSmonitoring and Larva.

With regard to the aim of the project, it was shown that the use of RV to validate commands was significantly more efficient than human verification. With that being noted, for the Verification system to be realistically implemented on the rover optimizations must be made in order for the RV system to require less processing power.

The results obtained during the evaluation stage show that the RV system would also be a viable option for verifying a set of commands the ground control team could set in advance and send to the rover to be executed. Rather than having a command system aboard the rover the ground control team could predetermine a set of commands to instruct the rover on where to move to next. Once the commands are transmitted to the rover the verification system would then sequentially validate all the commands and discard of any commands that could endanger the rover. Doing so could give the ground control team more control over what type of data is collected as well as freeing up some of the already very scarce processing power on board the rover since a command system would not be required for such an approach.

5.2 Critique & Limitations

Whilst the project can be considered a success with respect to the established aim there is still room for improvement. Namely in regards to the obstacle avoidance property, due to the limitations caused by certain tools on the simulated rover, primarily the front and back Hazcams. They were in a fixed position facing the ground and so any useful information for obstacle detection was not possibly extracted from these cameras. [1]

[1] I spoke to Mr Gatland about the case study and found he had similar issues when implementing obstacle detection of his own "Unfortunately getting obstacle data from the camera/depth sensors was by far the fiddliest part of my project! [...] I went back and forth between many different values in that

This greatly hindered the options of the ROStopics that could be used to collect data for obstacle avoidance. Consequently, the situation called for the use of a merged laser scanner which gave more accurate readings of obstacles around the rover but compromised the ability to locate the obstacle in relation to the rover and so proper obstacle avoidance could not be implemented.

Another limitation encountered was that a proper command control system was not implemented in this project. The main aim of this project was to create a verification system for the Curiosity rover and this was indeed implemented. However, had the command system been implemented in a proper manner rather than a few predefined commands the CPU usage of this system could have been obtained in order to better determine the feasibility of running all the systems on the rover's processor. Relating to this, the Curiosity rover was simulated and so statistics of the CPU usage could also not be gathered from the ROS system. Lacking both of these results we could not determine the feasibility of running the entire system implemented in this project on the actual Curiosity rover.

Building on this due to the limited time and resources available to implement this project we were unable to test the implementation on the real Mars rover on the actual Martian terrain. Had this been a possibility the system created could have been better tailored to the actual hardware the Curiosity rover makes use of updating the system to be better optimized for this.

5.3 Future Work

As discussed in the previous section improvements can be made to the obstacle detection implementation. The Curiosity rover model could be enhanced so the front and back Hazcams are better positioned for a better view of their surroundings. 2 new monitors could then be implemented in place of the merged scanner now collecting data from the Hazcams. These topics would give more detailed information on the rovers' surroundings consequently allowing for obstacle avoidance to be carried out.

The properties monitored in this project were relatively simple, the aim of this project was to show that RV could indeed be used to verify commands before they are executed by the system and this was shown to have been possible. Therefore, another improvement that could be made to the system is adding more properties to be monitored. For instance, monitors may be implemented to verify the power status of the rover, ensuring that if the rover hits a specific battery percentage it moves to a sunny

file [lasers.launch] until I found ones that worked well enough on the Gazebo worlds, so they're not very reliable."

area so that it may charge ensuring it does not fully drain its battery. Similarly, a monitor may be set up to watch the internal heat of the rover, ensuring that if the temperature hits a certain level it will move it into a shadier area so as not to cause any heat damage to any of its hardware.

Another future work that may be further worked on is implementing the command system. It could be properly created, implementing a system capable of generating specific commands for every property under monitoring. For instance under the given system, the camera information may be better utilised to detect objects in frame and use this information to create a navigation goals to move to a given position.

5.4 Related Work

In literature, numerous comparable works to ours may be found focusing on the verification of specific properties of a ROS robot. Y. S. Kirca et al. [12], implemented a similar approach focusing on security verification. Similar to our methodology they implemented an attacker system, a ROS system, and a verification system. Their main concern was protecting against unauthorized nodes from connecting to the system as an attacker. In contrast to our approach ROSmonitoring was used in the verification system to ensure it was accurately validating whether a connecting node was attacking or not. The verification made use of intrusion detection methods to warn the system of any possible attackers. From the conclusion they were able to confirm that their implementation indeed did help in reducing the human interaction required for protecting against attackers as well as cutting down on the V&V processing time. Their approach differed from ours in that our ROSmonitoring was implemented in the ROS system to collect data and pass it on to our verification system where they used it to verify that a connecting node was malicious.

In A. Ferrando et al. [1] they introduce the idea of a ROSmonitoring framework they created whereby a YAML file includes all the data required to generate a monitor in a ROS system. The generated monitor can either be injected in between the 2 nodes communicating or simply monitoring the data exchange between the 2 nodes. In their online monitoring implementation, the monitor collects data and passes on a verdict to an external Oracle. In contrast to our case, the monitor collects data and passes it on to our verification system to be used to validate the command to be executed. Their paper mostly focused on the scalability of using ROSmonitoring and seeing the overhead that was created. They found that the more monitors were added at the collection side of 2 communicating nodes the more overhead was created due to having to communicate with the Oracle once the monitor reached a verdict. Similarly, we also tested for over-

head however in our set up significantly fewer monitors were used and so was less of a concern by contrast to actually verifying that all commands executed were valid.

R. C. Cardoso et al. [13] had the most similar implementation. The focus of their literature was using RV to implement the verification of their autonomous verification system. Ensuring the rover would not execute any commands that would endanger it, along with ensuring that data was being collected. In contrast, to our approach, they made use of multiple verification techniques to ensure each property under surveillance had the best RV set up to ensure the final system ran as smoothly as possible.

5.5 Final Remarks

This project introduced the notion of using RV to verify rover commands before executing them. Although an idea that has already been explored it was re-implemented using Larva and ROSmonitoring. It has proven that the efficiency of verification through RV greatly triumphs over that of human verification.

References

[1] A. Ferrando, L. A. Dennis, D. Ancona, M. Fisher, and V. Mascardi, "Verifying and validating autonomous systems: Towards an integrated approach," in *Runtime Verification*, C. Colombo and M. Leucker, Eds., Cham: Springer International Publishing, 2018, pp. 263–281.

[2] M. Gatland, *Curiosity rover simulation*, Dec. 2016. [Online]. Available: https://markgatland.com/post/2022-curiosity/.

[3] E. Bartocci, Y. Falcone, A. Francalanza, and G. Reger, "Introduction to runtime verification," in Feb. 2018, pp. 1–33, ISBN: 978-3-319-75631-8. DOI: 10.1007/978-3-319-75632-5_1.

[4] G. Pace and C. Colombo, *Larva*. [Online]. Available: https://larva-turorials.netlify.app/about_larva.html?.

[5] *Ros -robot operating system*. [Online]. Available: https://www.javatpoint.com/tcp.

[6] "Ros -robot operating system," Tech. Rep., Dec. 2008. [Online]. Available: https://www.javatpoint.com/tcp.

[7] A. Ferrando, R. Cardoso, M. Fisher, D. Ancona, L. Franceschini, and V. Mascardi, "Rosmonitoring: A runtime verification framework for ros," in Dec. 2020, pp. 387–399, ISBN: 978-3-030-63485-8. DOI: 10.1007/978-3-030-63486-5_40.

[8] *Overview and usage of rqt*. [Online]. Available: https://docs.ros.org/en/humble/Concepts/About-RQt.html.

[9] *Ros - data display with rviz*. [Online]. Available: https://www.stereolabs.com/docs/ros/rviz/#:~:text=RVIZ%5C%20is%5C%20a%5C%20ROS%5C%20graphical,many%5C%20kinds%5C%20of%5C%20available%5C%20topics..

[10] D. Dobrijevic, *Distance to mars: How far away is the red planet?* Feb. 2022. [Online]. Available: https://www.space.com/16875-how-far-away-is-mars.html.

[11] C. Gartenberg, *Nasa's latest mars rover has the same processor as an imac from 1998*, Mar. 2021. [Online]. Available: https://www.theverge.com/tldr/2021/3/2/22309412/nasa-perseverance-mars-rover-processor-cpu-imac-1998.

[12] Y. Kirca *et al.*, "Runtime verification for anomaly detection of robotic systems security," *Machines*, vol. 11, p. 166, Jan. 2023. DOI: 10.3390/machines11020166.

[13] R. Cardoso, M. Farrell, M. Luckcuck, A. Ferrando, and M. Fisher, "Heterogeneous verification of an autonomous curiosity rover," in Aug. 2020, pp. 353–360, ISBN: 978-3-030-55753-9. DOI: 10.1007/978-3-030-55754-6_20.

Appendix A Test Results

Table A.1 Results of all tests run to collect CPU Usage in Verification System.

Test Cases	CPU Usage	
Test 1	Run 1	3.26
	Run 2	2.20
	Run 3	1.89
Test 2	Run 1	1.16
	Run 2	2.61
	Run 3	1.10
Test 3	Run 1	2.13
	Run 2	0.97
	Run 3	0.95
Test 4	Run 1	3.32
	Run 2	4.50
	Run 3	3.89
Test 5	Run 1	2.03
	Run 2	4.47
	Run 3	2.07

Table A.2 Results of all tests run to collect the Command Validation Latency in Verification System.

Test Case	Command Validation Latency Measures (seconds)											
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11
Test 1	Run 1	4.73	0.08	0.16	0.78	0.13	0.96	1.27	0.09	0.12	0.16	0.77
	Run 2	0.66	0.09	0.19	1.01	0.16	0.90	1.54	0.05	0.14	0.06	0.48
	Run 3	4.25	0.11	0.17	0.78	0.20	0.99	1.99	0.19	0.18	0.19	0.81
Test 2	Run 1	4.23	0.08	0.09	0.73	0.11	0.04	0.15	0.12	0.13	0.03	0.16
	Run 2	1.19	0.09	0.02	1.04	0.09	0.08	0.18	0.02	0.15	0.01	0.16
	Run 3	0.59	0.08	0.19	0.87	0.15	0.01	0.03	0.06	0.07	0.09	0.14
Test 3	Run 1	1.00	0.11	0.18	0.17	5.47	0.09	0.02	0.16	0.09	0.18	0.18
	Run 2	0.14	0.04	0.09	0.03	0.53	0.17	0.07	0.07	0.07	0.14	0.14
	Run 3	1.29	0.10	0.14	0.21	1.02	0.16	0.06	0.21	0.04	0.19	0.02
Test 4	Run 1	1.21	0.14	0.18	0.58	0.11	0.01	0.11	0.19	0.05	0.20	0.09
	Run 2	0.93	0.04	0.11	0.72	0.15	0.04	0.14	0.19	0.02	0.12	0.19
	Run 3	3.59	0.15	0.03	0.66	0.02	0.13	0.03	0.02	0.13	0.04	0.12
Test 5	Run 1	1.74	0.18	0.17	0.08	5.70	0.13	0.17	0.19	0.15	0.05	0.14
	Run 2	2.01	0.04	0.11	0.09	5.61	0.01	0.12	0.17	0.16	0.13	0.03
	Run 3	1.54	0.20	0.02	0.11	5.83	0.05	0.04	0.19	0.17	0.02	0.01

Appendix B Github Links

Case Study: https://github.com/mattea11/case_study.git

Verification System: https://github.com/mattea11/verification_system.git